

Contents

1. Abstract	Page 3
2. Problem Statement	Page 4
3. Proposed Solution	Page 5
4. Objectives	Page 6
5. Key Features	Page 7
6. Technology / Tech Stack	Page 8
7. System Architecture	Page 9
8. Implementation	Page 10
9. Results / Screenshots & Test Evidence	Page 11
10. Innovation	Page 14
11. Impact	Page 14
12. Future Scope	Page 15
13. References	Page 15

Reading guide. This report separates intended design from what has been built and verified. Every feature carries one of three labels: *Implemented and tested*, *Implemented, awaiting complete end-to-end verification*, or *Planned*. Test figures come from the project's own verification reports; nothing here is an unverified user outcome.

Section 1 | Abstract

Smart Share is a mobile-first platform that helps people discover others whose journeys are compatible with theirs, so that they can share a cab instead of booking separate ones. People leaving the same station, airport, event or business district at about the same time, heading in a similar direction, rarely know about each other. Smart Share lets a user post a ride request (pickup, destination, departure window, seats, and the maximum extra minutes they accept) and returns a ranked list of compatible requests with an explanation of why each was suggested.

The matching engine is location-aware and time-aware. It uses PostGIS to shortlist nearby pickups, then checks departure-window overlap, travel direction and seat capacity. It evaluates the valid pickup and drop-off orders for a pair, using OSRM road-network durations (with a clearly labelled approximate fallback), and keeps only pairings that stay inside each rider's detour limit. Pre-trip fare estimation (Phase 5B) implements the official Mumbai black-and-yellow taxi tariff with continuous linear calculation and Asia/Kolkata night surcharge windows. Pairwise cost allocation (Phase 5C) computes proportional shares based on solo baselines, conserving exact integer paise down to ■0.00 while strictly gating exact contribution disclosures behind DEC-013 bilateral affirmative consent.

The stack is React, TypeScript and Vite on the front end; Python, FastAPI and SQLAlchemy 2 on the back end; PostgreSQL 17 with PostGIS 3.3 hosted on Supabase; and Supabase Auth with backend ES256 JWT verification via JWKS.

Expected benefits are lower individual fares, better seat utilisation and fewer separate trips. These are expected benefits and have not been measured. Smart Share currently produces ranked suggestions and fare estimates; it does not book cabs, assign drivers or process payments.

Section 2 | Problem Statement

In dense cities such as Mumbai, many people start journeys from the same place, at nearly the same time, towards the same or neighbouring destinations. Yet each person typically books a separate cab, because there is no easy way to find out who else is going their way.

Typical situations:

- **Railway stations** (for example Dadar or CSMT): many commuters leave the same exit within minutes of each other.
- **Airports** (for example Mumbai T2 CSMIA): arriving passengers head to nearby suburbs in a similar direction.
- **Events and functions**: a college festival, concert, or wedding ends, and hundreds of guests leave at once.
- **Busy commercial areas** (for example BKC): office crowds travel to similar residential corridors in the evening.

Informal coordination through messaging groups is manual and unreliable. Even when two travellers find each other, they cannot easily judge whether sharing is practical: how far apart the pickups are, whether their departure times line up, or whether one rider would face an intolerable backtracking detour. Straight-line closeness is misleading in Mumbai; the project design notes show that two pickups at the same station can head in opposite directions (Dadar to Colaba versus Dadar to Borivali), and that water bodies (Mahim Bay, Mithi River) make nearby points far apart by road.

The consequences are unnecessary separate journeys, higher individual costs, underused vehicle capacity and added congestion near crowded venues.

Section 3 | Proposed Solution

Smart Share connects riders through ride requests. A user signs in, creates a request, and asks the system to find compatible requests from other users.

- **Create a ride request:** Choose a pickup and a destination (the current UI offers Mumbai landmark presets such as BKC, Airport T2, Powai, Lower Parel, Dadar and Andheri West).
- **Select a departure window:** State earliest and latest acceptable departure time.
- **State constraints:** Select seats needed (1–6) and maximum acceptable extra travel time (+0m to +60m).
- **Find matches:** The backend discovers candidates, filters them through spatial/temporal gates, and evaluates route sequences.
- **Review ranked suggestions:** View composite compatibility score, each rider's estimated detour, the pickup/drop-off sequence, and route accuracy notices.
- **Fare estimation & DEC-013 consent:** Inspect the pre-trip fare estimate and privacy-safe economic status; affirmatively opt-in to reveal exact proportional contributions.



Figure 1. End-to-end user journey in the Smart Share application.

How candidates are found: The engine shortlists active, unexpired requests from other users within 2 km of the pickup, with overlapping departure windows and combined seats of at most four. It then rejects pairs travelling in clearly different directions (>90° divergence), evaluates the valid pickup/drop-off orders using road-network durations, and discards any order where either rider's extra time exceeds their own stated limit. Survivors are scored and the top five are returned.

Scope note: Automatic cab dispatch, driver assignment, and payment processing are explicitly out of MVP scope.

Section 4 | Objectives

Technical Objectives

- Discover compatible passengers with location-aware spatial queries (PostGIS geography, GiST index).
- Apply time-aware matching through departure-window intersection.
- Evaluate direction and destination compatibility, rejecting backtracking pairings.
- Evaluate valid pickup/drop-off orders and keep each rider's detour within their stated limit.
- Provide deterministic, statutory pre-trip fare estimation using the approved Mumbai taxi tariff model.
- Allocate shared costs proportionally to solo baselines, conserving exact paise down to ■0.00.
- Enforce DEC-013 bilateral consent to protect passenger privacy and prevent unilateral financial inference.
- Maintain a scalable foundation: modular services, replaceable routing adapters, Alembic migrations, and automated tests.

User-Impact Objectives

- Make it easy to find fellow travellers, reducing reliance on duplicate solo cab journeys.
- Improve vehicle seat occupancy during high-demand congestion hours.
- Minimise detour delays through strict passenger-configured detour ceilings.
- Offer an accessible, mobile-first experience with intuitive landmark presets and clear status feedback.
- Help passengers clearly understand whether pooling saves money through honest, non-manipulated economic indicators.

The user-impact objectives represent engineering design goals. Pilot conversion and savings targets remain subject to hosted user trials.

Section 5 | Key Features

Feature status is verified against repository code, test suites, and database migrations. *Tested* denotes automated test verification, plus live database checks where noted.

Feature	Planned	Implemented	Tested	Status
Supabase Auth with backend JWT verification (ES256/JWKS)	Yes	Yes	Auth test suite (18 tests)	**Implemented and tested**
User profiles (retrieve/update)	Yes	Yes	API tests	**Implemented and tested**
Ride request create, list, view, edit, cancel; ownership checks	Yes	Yes	API + model tests; UI tests	**Implemented and tested**
Landmark presets, departure window, seats, detour preference	Yes	Yes	UI component tests	**Implemented and tested**
PostGIS candidate filtering (2 km radius, window overlap, LIMIT 20)	Yes	Yes	Unit tests; live PostGIS run (rollback)	**Implemented and tested; browser E2E pending**
Direction and seat-capacity checks	Yes	Yes	Unit tests	**Implemented and tested**
Pairwise stop-order evaluation & per-rider detour limits	Yes	Yes	Unit tests	**Implemented and tested**
Weighted scoring, ranking (top 5), and match explanations	Yes	Yes	Unit, API and UI tests	**Implemented; hosted 2-user E2E pending**
OSRM road-network durations and approximate fallback	Yes	Yes	Adapter and fallback tests	**Implemented and tested (best-effort)**
Statutory Mumbai taxi fare estimation (Phase 5B)	Yes	Yes	Tariff, pulse, night-window tests	**Implemented and tested**
Solo-proportional fare allocation & DEC-013 consent (Phase 5C)	Yes	Yes	12 invariant tests; paise conservation	**Implemented (local verified; hosted pending)**
Match acceptance / decline, ride groups, proposal expiry	Yes	No	-	Planned (Phase 7)
Cab booking, payments, live dispatch, vehicle tracking	Future	No	-	Out of MVP scope

Table 1. Feature-status matrix (reflecting verified Phase 5B/5C implementation).

Scoring model (as implemented):

Composite score = 30% pickup proximity + 30% destination similarity & direction + 20% departure-time compatibility + 20% individual detour headroom. Seat capacity is a hard eligibility gate, not a scoring factor. Ties are broken by lower total detour, then smaller pickup separation.

Section 6 | Technology / Tech Stack

Frontend: React 18 · TypeScript · Vite · Vitest · Tailwind CSS

Backend: Python 3.14 · FastAPI · SQLAlchemy 2 (async) · Pydantic v2

Database: PostgreSQL 17.11 · PostGIS 3.3.7 · Supabase (hosted) · Alembic migrations

Identity & Auth: Supabase Auth · ES256 asymmetric JWT verification via JWKS

Routing: OSRM Table API adapter · Calibrated tortuosity fallback (geodesic × 1.40, 22 km/h)

Quality & CI: Pytest (backend) · Vitest (frontend) · Ruff · Oxlint · TypeScript compiler

Figure 2. Technology stack, limited to components documented as implemented.

Technology	Purpose in Smart Share	Why it fits
React, TypeScript, Vite	Mobile-first UI, forms, and match results	Fast dev builds, typed props, rich ecosystem
Python, FastAPI	REST API under /api/v1 and matching engine	High async throughput, clean dependency injection
Pydantic v2	Request/response schema validation & DEC-013 redaction	Guarantees private fields are omitted from wire payloads
PostgreSQL + PostGIS	Geography(Point, 4326) storage and spatial queries	ST_DWithin with GiST index accelerates pickup filtering
Supabase (hosted)	Managed PostgreSQL, PostGIS, and user authentication	Reliable hosted cloud DB without DevOps overhead
Alembic	Versioned database schema migrations (c5d013c0a501)	Deterministic, additive, reviewable schema evolution
Supabase Auth / JWKS	User sign-in; backend verifies ES256 bearer tokens	Cryptographically derived identity, zero passwords stored
OSRM (+ Fallback)	Road-network travel durations for detour calculations	Zero-credential routing with graceful offline degradation
Pytest, Ruff, Oxlint, tsc	Automated unit/API tests, static linting, and type checking	Prevents regressions and enforces strict code cleanliness

Table 2. Technologies and architecture rationale.

Section 7 | System Architecture

Primary Data Flow:

User (Mobile Browser) → React Client → Supabase Auth (Sign-in / JWT) → FastAPI Backend (/api/v1) → JWT Verification (JWKS ES256) → Matching Engine → PostGIS (Spatial Discovery) → OSRM Adapter (Road Network Detours) → Fare Estimator (Phase 5B) → Proportional Allocation (Phase 5C) → DEC-013 Consent Filter → Scoped Proposal JSON → React UI.

Figure 3. System architecture and primary end-to-end data flow.

Backend Authoritativeness & Decoupled Routing: The frontend never determines match feasibility or cost allocation; the backend is authoritative. The routing adapter sits behind a provider interface, allowing OSRM, OpenRouteService, or the calibrated fallback to be swapped transparently without modifying matching heuristics. Match proposals are computed dynamically on demand; cancelling or expiring a request automatically excludes it from future queries.

1. Spatial Discovery	PostGIS ST_DWithin <= 2,000m on pickup points with GiST index.
2. Temporal Overlap	Closed-interval departure window intersection [start, end].
3. Direction Gate	Heading bearing difference <= 45° (or <= 90° if destinations within 3km).
4. Capacity Gate	Combined seats requested <= 4 (hard vehicle capacity gate).
5. Sequence Evaluation	Evaluates 4 genuinely shared stop orders out of 6 precedence permutations.
6. Detour Verification	Verifies individual rider extra time stays within stated max_extra_minutes.
7. Fare Estimation (5B)	Computes statutory Mumbai pre-trip fare estimates for shared and solo journeys.
8. Allocation & DEC-013 (5C)	Allocates proportional shares; gates exact contribution behind bilateral consent.

Figure 4. Matching engine execution pipeline (Phases 4B, 5B, and 5C).

Section 8 | Implementation

Phase 0: Planning (completed)	**Phase 1:** Foundation (completed)	**Phase 2:** Database & Auth (completed)
Phase 3: Ride Requests (completed)	**Phase 4:** Matching Engine (completed)	**Phase 5:** Fares & Allocation (5B/5C implemented)
Phase 6A: Consent UX (started)	**Phase 7:** Group Lifecycle (planned)	**Phases 8-9:** Production Deploy (planned)

Figure 5. Implementation milestone roadmap (Phase 5 completed locally).

Milestone Summaries:

- **Phase 0–3:** Completed foundational architecture, PostgreSQL/PostGIS schemas, Alembic migrations, Supabase Auth ES256 verification, and authenticated CRUD ride request management.
- **Phase 4 (Matching Engine):** Implemented spatial shortlisting, 90° direction gating, pairwise stop-order enumeration, detour scoring, and ranked match explanation payloads.
- **Phase 5B (Fare Estimation):** Implemented statutory Mumbai taxi tariff (■33 base fare for 1.5 km, ■21.90/km), Asia/Kolkata night surcharge window (00:00–05:00 IST), and conservative route accuracy tiers (ROAD_ROUTE, MATRIX_SCREENING, CALIBRATED_FALLBACK).
- **Phase 5C (Fare Allocation & Privacy):** Implemented solo-proportional allocation (DEC-008), deterministic Half-Up integer-paise conservation, economic classification (POSITIVE, BREAK_EVEN, NEGATIVE, CONDITIONAL), and DEC-013 bilateral affirmative consent persistence in match_disclosure_consent_events.

Key Technical Challenges & Resolutions:

Challenge	Resolution
Straight-line distance misleads in Mumbai (water bodies, opposite headings from Dadar/CSMT)	Direction gate (hard reject >90°) and OSRM road-network travel durations for detours.
Precedence permutations where riders never share cab	Engine enumerates 6 orders and discards the 2 non-sharing sequences (P1-D1-P2-D2, P2-D2-P1-D1).
Public routing service rate limits and downtime	1.05s request spacing, bounded candidate batching, and calibrated geodesic fallback.
PostGIS returned raw hex EWKB for geography columns	Queries select ST_AsText; parser also decodes raw WKB/EWKB as fallback.
Unilateral financial inference risk (DEC-013)	Bilateral affirmative consent required before exact contribution disclosure; solo baselines remain redacted.

Table 3. Key engineering challenges and technical resolutions.

Section 9 | Results / Screenshots

This section presents visual verification evidence from the running Smart Share application. Screenshots captured from the running application are distinguished from pending hosted items. In accordance with academic verification standards, screenshot placeholders remain for unsupplied views.

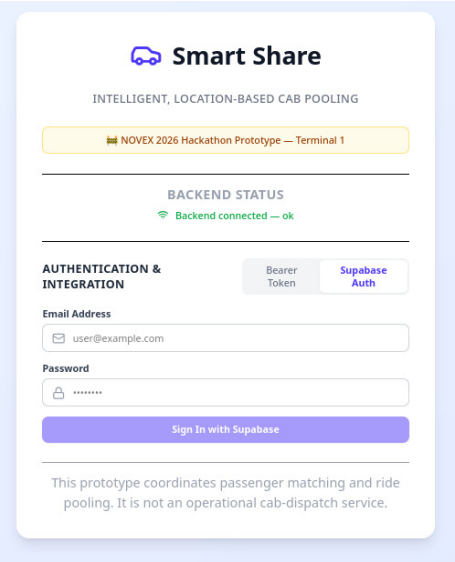


Figure 9. User authentication interface showing Supabase Auth integration and backend health status.

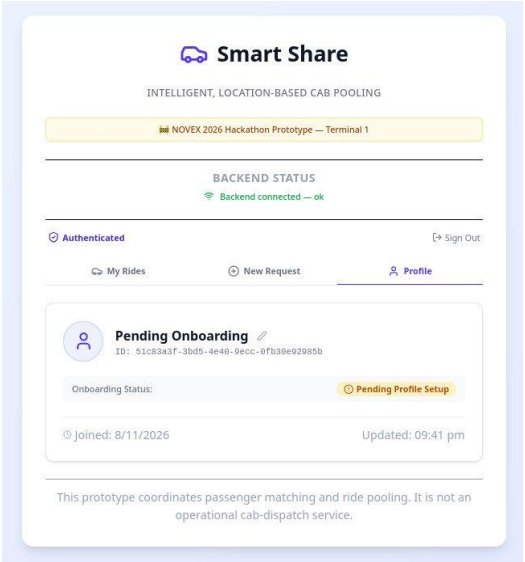


Figure 13. Authenticated user profile and onboarding setup interface with development UUID.

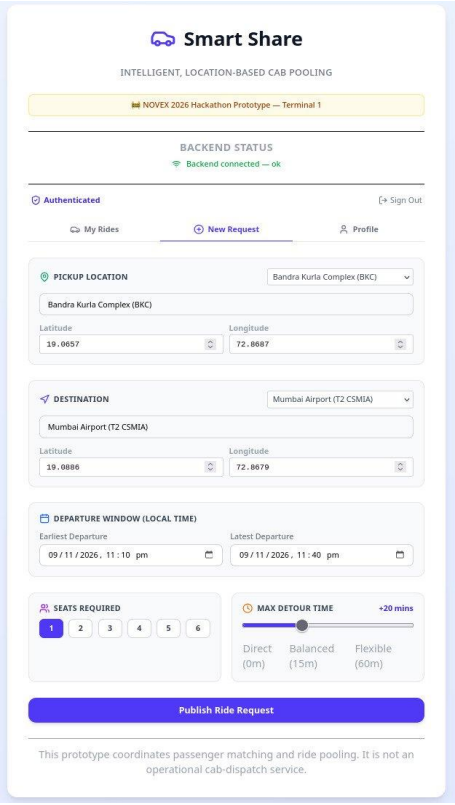


Figure 10. Ride request creation interface with landmark presets, departure window, and detour slider.

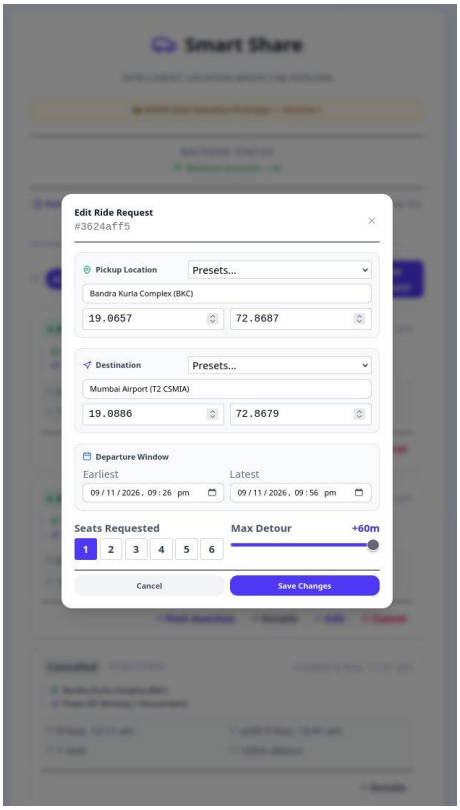


Figure 12. Ride request details and edit interface for modifying active departure constraints.

Section 9 | Results / Screenshots (Cont.)

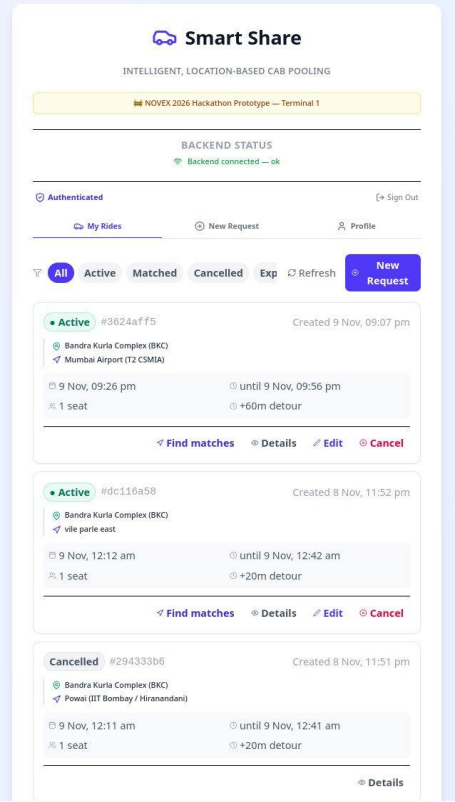


Figure 11. Active ride request listing with status badges and on-demand 'Find matches' action.

[SCREENSHOT PENDING — HOSTED ACCEPTANCE RUN]

Figure 14 & 15. Match discovery proposals & DEC-013 bilateral fare disclosure.

To be captured during the upcoming live hosted two-user acceptance test between Passenger A and Passenger B on the deployed Supabase / Render / Vercel infrastructure.

Figure 14. Ranked match proposals with statutory fare estimates and DEC-013 consent (Pending).

Note on Unsupplied Screenshots: Genuine application captures have been inserted for Authentication (Fig. 9), Request Creation (Fig. 10), Request Listing (Fig. 11), Request Edit (Fig. 12), and User Profile (Fig. 13). Screenshots for the unauthenticated landing homepage (Placeholder 1) and live two-user consented match results (Fig. 14) are pending live hosted acceptance verification and are truthfully designated as pending.

Documented Testing Results & Acceptance

Evidence Layer	Verified Result	Verification Source
Full Backend Pytest Suite	175 passed, 1 skipped (hosted PostGIS sandbox restriction) in 12.70s	Clean baseline 536c92a (app/tests)
Fare & Matching Tests	51 passed in 0.66s (covering 12 mathematical invariants, tariff & allocation)	test_fares.py & test_matching_algorithms.py
Live PostGIS Query Test	1 passed in 0.49s against Supabase PostgreSQL inside rolled-back transaction	test_matching_postgis.py
Frontend Vitest Suite	32 passed across 7 test files in 3.48s	frontend/src/features/ride-requests
Static Quality & Linters	Ruff (0 errors, 74 files clean), Oxlint (0 errors), tsc --noEmit (0 errors)	CI quality check suite
Production Client Build	Vite production bundle built successfully in 0.39s (125 kB gzip)	npm run build
Database Migration	Migration c5d013c0a501 verified on Supabase development DB (ifdmbwjaysspvysoxfpl)	Alembic current revision

Table 4. Comprehensive test execution evidence across backend, frontend, and database.

Verification status, stated transparently:

Phase 5C Verdict: CONDITIONAL PASS. All mathematical allocation algorithms, non-negative bounds, proportional solo weights, deterministic integer-paise conservation ($C_A + C_B \equiv S$), and DEC-013 server-side redaction filters have passed automated unit and component testing. However, a live two-user acceptance test between two separate authenticated commuters on deployed infrastructure (Render/Vercel/Supabase) has not yet been executed. Therefore, complete hosted acceptance remains conditional.

Source Tree State Notice: The Git repository HEAD commit (eef01c7) contains unresolved merge conflict markers across 6 source files that must be reconciled before deployment to staging/production.

Section 10 | Innovation

Smart Share's contribution is not simply splitting a cab fare. It is deciding which strangers can practically share a journey, and showing why. It combines several checks that simple proximity matching lacks:

- **Location-aware spatial discovery:** PostGIS geography indexing with 2 km radius shortlisting.
- **Time compatibility:** Closed-interval overlap of departure windows prevents impossible meetups.
- **Direction gating:** Rigid bearing checks ensure riders heading in opposite directions are never paired.
- **Detour-aware sequence optimization:** Evaluates 4 shared stop orders against individual detour tolerances.
- **Statutory Mumbai taxi tariff:** Accurate pre-trip estimation using official government rate models.
- **Solo-proportional fair allocation:** Weighting cost contributions by individual solo baselines with zero-paise residual drift.
- **DEC-013 Bilateral Consent Privacy:** Cryptographic snapshot binding prevents unilateral economic profiling.

Traditional Approach: Separate bookings → Duplicate cab journeys → Higher individual costs → Empty seats → Street congestion.

Smart Share Approach: Intelligent spatial/temporal matching → Feasible pooled routes → Statutory fare estimation → Fair proportional allocation → Bilateral privacy protection.

Figure 6. Traditional cab booking versus Smart Share intelligent pooling.

Section 11 | Impact

EXPECTED impact only. No user adoption, savings or emissions figures have been measured.

Economic	Environmental	Social	Operational
Potential lower travel outlays; improved cost efficiency per passenger.	Fewer duplicate trips, reduced fuel consumption and vehicle tailpipe emissions.	Easier coordination among commuters leaving crowded venues.	Higher vehicle occupancy and reduced bottleneck congestion at hubs.

Figure 7. Expected project impact dimensions (subject to future live deployment trials).

Section 12 | Future Scope

All items below represent future development roadmap milestones:

Priority	Milestone Description	Roadmap Link
Near term	Reconcile Git merge conflicts and complete authenticated hosted two-user acceptance testing.	Phase 5C Acceptance
Next	Match acceptance / decline, ride group confirmation, proposal expiry, and safe seat allocation.	Phase 7
Next	Interactive Leaflet map route visualization and web push status notifications.	Phases 6–7
Later	Multi-passenger matching optimization (pooling groups of 3 or 4 riders).	Matching Extension
Later	Production deployment, traffic-aware routing adapter, and realistic load testing.	Phases 8–9
Out of MVP	Commercial cab-provider API integration, payment gateway processing, and real-time taxi dispatch.	Future Commercial

Table 5. Future development scope and roadmap priorities.

Section 13 | References

- React documentation: https://react.dev
- Vite build tool: https://vite.dev
- TypeScript documentation: https://www.typescriptlang.org/docs/
- FastAPI framework: https://fastapi.tiangolo.com
- Pydantic v2 documentation: https://docs.pydantic.dev
- SQLAlchemy 2.0: https://docs.sqlalchemy.org
- Alembic database migrations: https://alembic.sqlalchemy.org
- PostgreSQL 17: https://www.postgresql.org/docs/
- PostGIS spatial extension: https://postgis.net/documentation/
- Supabase documentation: https://supabase.com/docs
- OSRM Table service API: https://project-osrm.org/docs/v5.24.0/api/#table-service
- pytest testing framework: https://docs.pytest.org
- Vitest frontend testing: https://vitest.dev
- Ruff Python linter: https://docs.astral.sh/ruff/
- Internal Project Documentation: Project Charter; SRS/PRD; System Architecture; Database & API Contracts; Roadmap; Phase 4B Matching Reports; Phase 5B Fare Estimation Design; Phase 5C Proportional Allocation & Privacy Reports.